

CREDIT CARD APPROVAL PREDICTION

An End-to-End Machine Learning Project Report

Submitted as part of the Smart Bridge Internship Program
Artificial Intelligence and Machine Learning

Submitted by

P.DHARSHINI – AP24110011138

V.HARIVEER – AP24110010852

G.KARTHIK VASAN – AP24110011166

K.V.SUDHAKAR REDDY – AP24110012124

SRM UNIVERSITY AP

CSE

2024-2028

Table of Contents

Content	Page No.
1. Introduction	3
1.1 Project Overview	3
1.2 Purpose	3
2. Ideation Phase	3
2.1 Problem Statement	3
2.2 Empathy Map Canvas	4
2.3 Brainstroming	4
3. Requirements Analysis	5
3.1 Customer Journey Map	5
3.2 Solution Requirement	5
3.3 Data Flow Diagram	6
3.4 Technology Stack	6
4. Project Design	7
4.1 Problem–Solution Fit	7
4.2 Proposed Solution	7
4.3 Solution Architecture	8
5. Project Planning & Scheduling	8
5.1 Project Planning	8
6. Functional and Performance Testing	9
6.1 Performance Testing	9
7. Results	10
7.1 Output Screenshots	10
8. Advantages & Disadvantages	13
9. Conclusion	14
10. Future Scope	14
11. Appendix	15

1. Introduction

1.1 Project Overview

Credit Card Approval Prediction is an end-to-end machine learning project that predicts whether a credit card applicant is likely to be Approved or Rejected. The project covers the full ML lifecycle, starting from raw data collection through preprocessing, exploratory data analysis, model training and comparison, and finally deployment through a Flask-based web application that supports both single-applicant and batch (CSV) predictions.

The system is built around two source files, `application_record.csv` and `credit_record.csv`, which are merged on a common applicant ID. A binary approval target is derived from the applicant's historical repayment status, and this label is used to train and compare several classification models. The best-performing model is saved and served through a web interface that allows a user to enter applicant details and instantly receive a prediction along with an approval probability.

1.2 Purpose

Banks and credit card providers receive large volumes of applications and need a consistent, data-driven way to screen applicants for credit risk. Manually reviewing every application is slow, inconsistent, and prone to human error. The purpose of this project is to demonstrate how a machine learning pipeline can automate this initial screening step by learning patterns from historical applicant and repayment data, and to package that pipeline as a usable web application.

- Provide a working prototype that mentors and reviewers can use to see the complete ML lifecycle.
- Give students hands-on experience with data preprocessing, feature engineering, model comparison, and deployment.
- Offer a reference tool that a bank analyst could use as a starting point for an approval-risk screening system.

2. Ideation Phase

2.1 Problem Statement

Banks and credit card providers receive many applications and need a consistent way to estimate whether an applicant is likely to be a low-risk or high-risk customer. Relying purely on manual review does not scale, and inconsistent judgment calls between reviewers can lead to unfair or unpredictable outcomes. There is a need for a system that uses an applicant's demographic, financial, and employment information, combined with historical repayment behaviour, to predict whether a new application should be approved or rejected.

2.2 Empathy Map Canvas

The empathy map below captures the perspective of the primary user group considered during ideation: a bank credit-risk analyst evaluating incoming applications.

Quadrant	Observations
Says	"I need a faster way to screen applications." "I want consistent decisions across all analysts." "I don't fully trust a black-box decision."
Thinks	Worries about approving a high-risk applicant or rejecting a good one. Wants a tool that explains, at least at a high level, why a decision was made.
Does	Manually cross-checks income, employment, and repayment history. Repeats similar checks for every application. Keeps informal notes on risky patterns.
Feels	Time pressure from application backlog. Frustration with repetitive manual checks. Relief when a tool can pre-screen obvious cases.
Pains	Slow manual review, inconsistent decisions, difficulty handling large batches of applications, risk of human error.
Gains	Faster turnaround time, consistent risk scoring, an approval probability to support the final decision, downloadable reports for record-keeping.

2.3 Brainstorming

The team brainstormed the overall solution approach before building the pipeline. Key ideas that shaped the final solution:

- Load the application and credit repayment datasets and merge them using the applicant ID.
- Convert the multi-class repayment status into a single binary approval target.
- Perform data cleaning, exploratory data analysis, and feature engineering (age, years employed, income category, financial stability score, risk category, etc.).
- Train and compare multiple classification models rather than relying on a single algorithm.
- Save the best-performing model and metadata for reuse without retraining.
- Wrap the model in a Flask web application that supports single-applicant prediction, batch CSV prediction, prediction history, and downloadable reports.
- Target users: mentors reviewing project evidence, students demonstrating the ML lifecycle, and bank analysts wanting a prototype screening tool.

3. Requirement Analysis

3.1 Customer Journey Map

The table below traces the journey of a user (applicant or analyst) through the web application, from landing on the home page to receiving and downloading a prediction.

Stage	User Action	System Response	User Emotion
Discovery	Opens the home page of the web app	Displays project overview and navigation to prediction, model info, and about pages	Curious
Access	Signs up or logs in (including Google login)	Validates credentials and creates a session	Slightly cautious
Input	Fills the single-applicant prediction form or uploads a batch CSV	Validates required fields / CSV format before submission	Focused
Prediction	Submits the form or file	Loads the saved model, generates Approved/Rejected result with approval probability	Anticipation
Review	Views the result page or batch results table	Shows prediction, probability, and risk level; strong or risky profiles are highlighted	Relief or concern
Record-keeping	Downloads result as PDF or Excel; views prediction history	Generates downloadable report; stores entry in local SQLite history	Satisfied

3.2 Solution Requirement

Functional Requirements

- The system must accept applicant details through a web form.
- The system must allow CSV upload for batch prediction.
- The system must validate required input fields before prediction.
- The system must load a trained machine learning model from models/best_model.pkl.
- The system must return an Approved or Rejected result with an approval probability.
- The system must keep prediction history in local SQLite storage during app use.
- The training workflow must compare Logistic Regression, Decision Tree, Random Forest, and XGBoost, in addition to SVM, KNN, Naive Bayes, and Gradient Boosting.

Non-Functional Requirements

- The project should run locally with a single pip install -r requirements.txt command.
- Secrets must stay outside version control by using a .env.example file instead of committing .env.
- The app should provide clear, user-friendly validation messages.
- Reports and plots should be reproducible simply by re-running train.py.

3.3 Data Flow Diagram

The diagram below shows how data moves through the system, from the applicant's input to the final downloadable report.

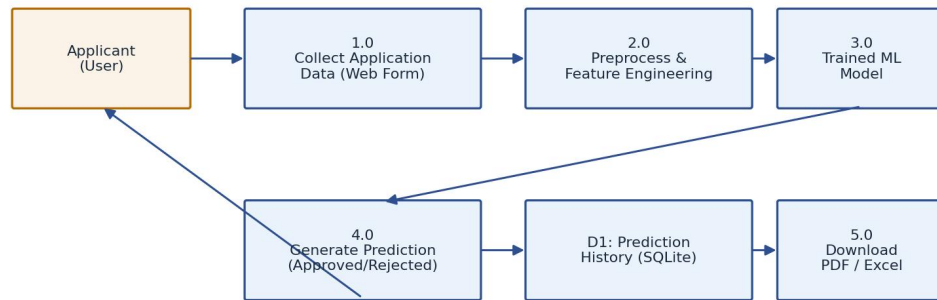


Figure 1: Data Flow Diagram (Level 1) of the Credit Card Approval Prediction system

3.4 Technology Stack

Layer	Technology
Programming Language	Python 3
Web Framework	Flask 3.0.3
Data Handling	pandas 2.2.3, numpy 2.0.2
Machine Learning	scikit-learn 1.5.2, XGBoost 2.1.3
Model Persistence	joblib 1.4.2
Explainability	SHAP 0.46.0
Visualization	matplotlib 3.9.4, seaborn 0.13.2, plotly 5.24.1
Reporting	ReportLab 4.2.5 (PDF), openpyxl 3.1.5 (Excel)
Database	SQLite (prediction history)
Testing	pytest 8.3.4
Deployment	Gunicorn 23.0.0, Procfile, Dockerfile, runtime.txt
Frontend	HTML templates (Jinja2) with custom CSS

4. Project Design

4.1 Problem – Solution Fit

Problem: manual, inconsistent credit-risk screening cannot scale with growing application volumes and produces variable outcomes across reviewers. Solution: a trained classification pipeline that learns from historical applicant and repayment data and produces a consistent, reproducible

Approved/Rejected decision with an accompanying probability score, exposed through an accessible web interface. The fit is validated by the ability to process both single applications interactively and larger batches through CSV upload, mirroring how an analyst would handle individual cases as well as bulk queues.

4.2 Proposed Solution

The proposed solution follows the standard ML lifecycle, implemented across dedicated modules:

- Data Collection: `application_record.csv` and `credit_record.csv` act as the raw inputs.
- Preprocessing (`preprocessing.py`): merges the two datasets on ID, builds the binary target from repayment status, cleans missing occupation values, caps numeric outliers, and engineers features such as age, years employed, income category, family income ratio, financial stability score, and risk category.
- Model Training (`train.py`): trains and compares eight classification models and records accuracy, precision, recall, F1 score, ROC-AUC, confusion matrix, and cross-validation F1 for each.
- Model Selection: the best-performing model and its metadata are saved to the `models/` directory using `joblib`.
- Prediction Service (`predict.py`): reusable logic that loads the saved model and produces predictions for new applicant data.
- Web Application (`app.py`): a Flask app exposing single prediction, batch prediction, authentication (including Google login), prediction history, and PDF/Excel export.

4.3 Solution Architecture

The architecture below summarizes how the components connect, from raw data to the deployed web application.

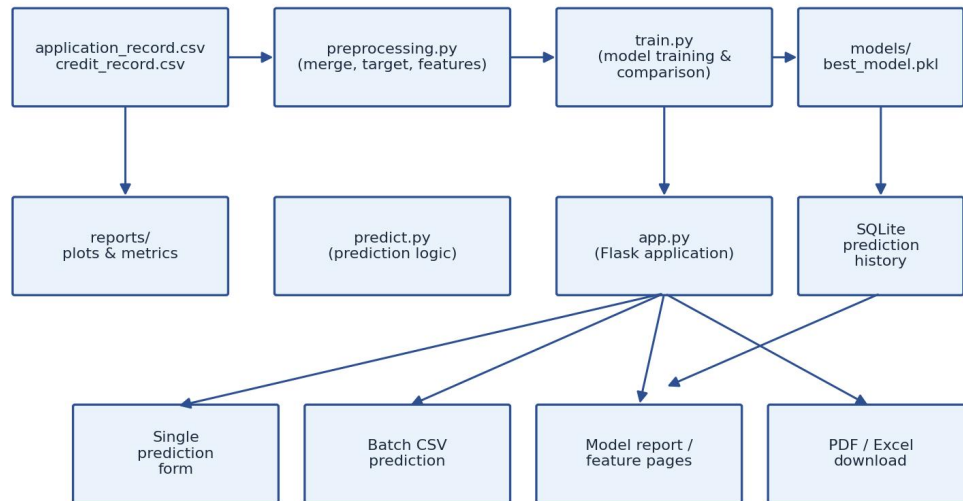


Figure 2: Solution architecture of the Credit Card Approval Prediction system

5. Project Planning & Scheduling

5.1 Project Planning

The work was organized into epics, each with a clear scope and evidence of completion:

Epic	Scope	Status	Evidence
Epic 1: Data Collection	Add application and credit record CSV files	Done	data/
Epic 2: EDA	Load data, inspect shape, missing values, target distribution, and plots	Done	notebooks/EDA.ipynb
Epic 3: Preprocessing	Merge data, convert target, clean records, engineer features	Done	preprocessing.py
Epic 4: Model Building	Train and compare multiple ML models	Done	train.py, reports/model_comparison.csv
Epic 5: Flask App	Build prediction UI and connect saved model	Done	app.py, templates/, static/
Epic 6: Testing	Add automated and manual test evidence	Done	tests/, Project Testing/
Epic 7: Documentation	Explain setup, workflow, and outputs	Done	README.md, Project Documentation/
Epic 8: Demonstration	Show the working app	In progress	Project Demonstration/

Execution plan followed by the team:

- 1. Prepare the dataset.
- 2. Explore and understand the data.
- 3. Convert repayment status into a binary target.
- 4. Clean and engineer features.
- 5. Train candidate models.
- 6. Select and save the best model.
- 7. Build Flask prediction screens.
- 8. Test the app and document outputs.
- 9. Record demo video and finalize screenshots.

6. Functional and Performance Testing

Automated tests are stored in `tests/test_app.py` and cover CSV file validation, form validation, credit-rule overrides for underage/student/unemployed/low-income/strong-profile applicants, Excel download generation, prediction history deletion and clearing, login/signup behaviour, and Google login configuration. Tests are run using `pytest`.

6.1 Performance Testing

The current model comparison output, generated by running `train.py`, is summarized below.

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
KNN	0.7500	0.7500	1.0000	0.8571	0.4167
Logistic Regression	0.6250	1.0000	0.5000	0.6667	0.5000
Support Vector Machine	0.3750	0.6000	0.5000	0.5455	0.5000
Random Forest	0.3750	0.6000	0.5000	0.5455	0.3333
XGBoost	0.3750	0.6000	0.5000	0.5455	0.0000
Decision Tree	0.2500	0.5000	0.3333	0.4000	0.1667
Naive Bayes	0.2500	0.5000	0.3333	0.4000	0.1667

Note: the bundled dataset is a small sample used to demonstrate the full pipeline end to end. Because the test split is small, metrics can shift noticeably between runs; the testing evidence therefore focuses on pipeline correctness and reproducible app behaviour rather than production-grade accuracy claims.

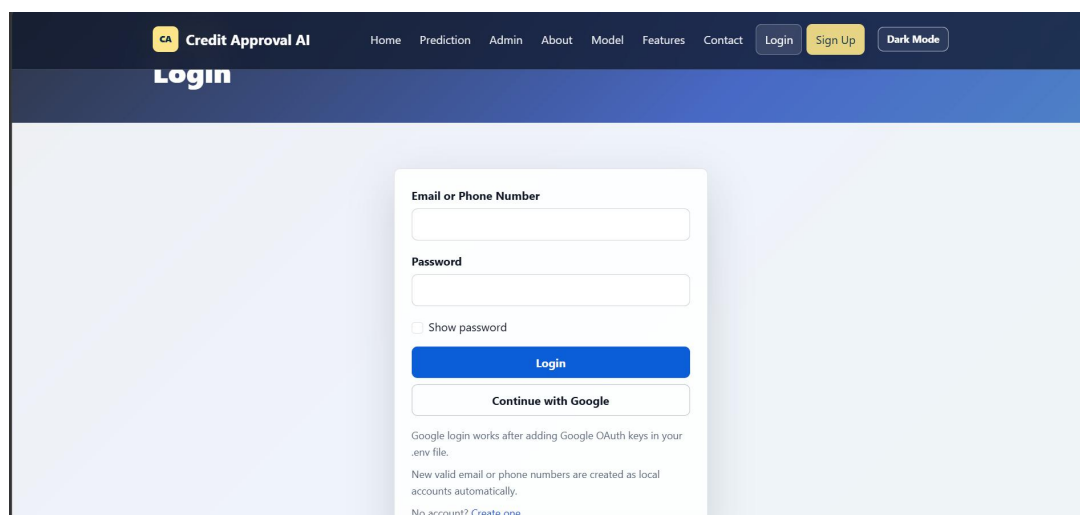
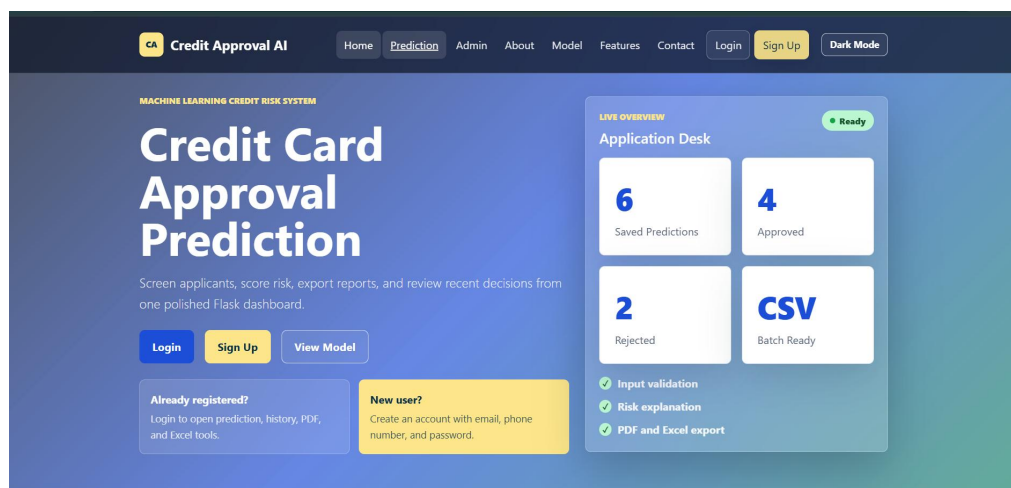
Test Case	Input Summary	Expected Output	Actual Output
Strong working applicant	Age 35, income 250000, manager, owns car and real estate	Approved	Covered by automated tests

Test Case	Input Summary	Expected Output	Actual Output
Underage applicant	Age 12	Rejected	Covered by automated tests
Student applicant	Income type Student	Rejected	Covered by automated tests
Unemployed applicant	Income type Unemployed	Rejected	Covered by automated tests
Low-income applicant	Income below 100000	Rejected	Covered by automated tests
Invalid batch file	Upload non-CSV file	Rejected by validation	Covered by automated tests

7. Results

The trained pipeline produces standard classification evaluation artifacts — a confusion matrix, an ROC curve, and a precision-recall curve — generated automatically in reports/plots/ whenever `train.py` is run.

7.1 Output Screenshots



CA Credit Approval AI

[Home](#) [Prediction](#) [Admin](#) [About](#) [Model](#) [Features](#) [Contact](#) [Login](#) [Sign Up](#) [Dark Mode](#)

First Name

Last Name

Email

Phone Number

Password

Confirm Password

☐ Show password

Create Account

CA Credit Approval AI

[Home](#) [Prediction](#) [Admin](#) [About](#) [Model](#) [Features](#) [Contact](#) [Logout](#) [Dark Mode](#)

01 Applicant Details

Use realistic customer values for the most useful prediction.

First Name

Last Name

Phone Number

Gender

Age

Annual Income

Family Members

Children

Years Employed

Education

Occupation

Housing Type

Income Type

RESULT PREVIEW

Waiting for input

Submit the customer information to see approval probability, risk level, confidence, and recommendations.

Approval probability

Decision reasons

PDF and Excel report

Batch Prediction

Upload a CSV file only. CSV means a spreadsheet saved as comma-separated values, usually from Excel or Google Sheets.

Resume, PDF, DOC, and image files are not used in this credit approval model. For one person, fill the form above. For many people, use a CSV with the same columns as the sample file.

CA Credit Approval AI

[Home](#) [Prediction](#) [Admin](#) [About](#) [Model](#) [Features](#) [Contact](#) [Logout](#) [Dark Mode](#)

TIME	NAME	INCOME	EDUCATION	PREDICTION	PROBABILITY	RISK	ACTION
2026-07-03T18:41:28	Harshini N	550000	Higher education	Approved	79.5%	Low Risk	Delete
2026-07-03T18:40:31	Ashwanth ch	250000	Higher education	Approved	57.75%	Moderate Risk	Delete
2026-07-03T18:38:21	Pavan Kumar K	30000	Incomplete higher	Rejected	28.0%	High Risk	Delete
2026-07-03T18:36:11	Sravanthi s	300000	Higher education	Approved	77.5%	Low Risk	Delete
2026-07-03T18:34:25	Maithree P	500	Lower secondary	Rejected	22.0%	High Risk	Delete
2026-07-02T19:08:17	Mahi k	1000000	Secondary / secondary special	Approved	95.0%	Low Risk	Delete

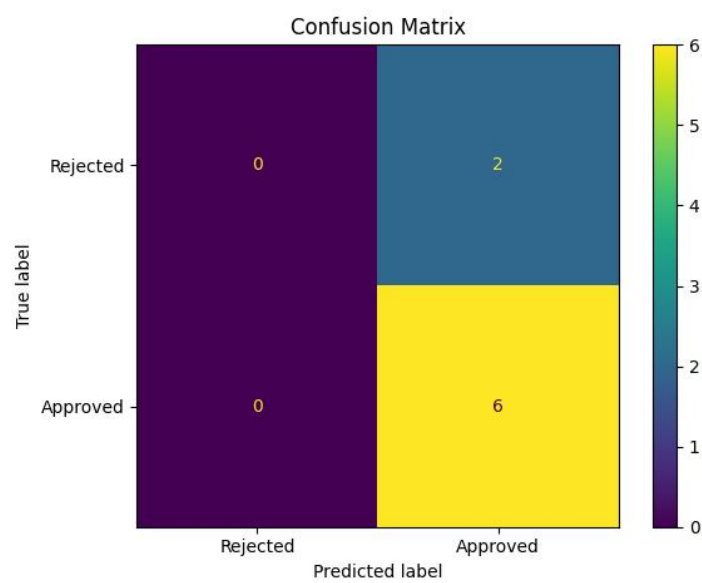
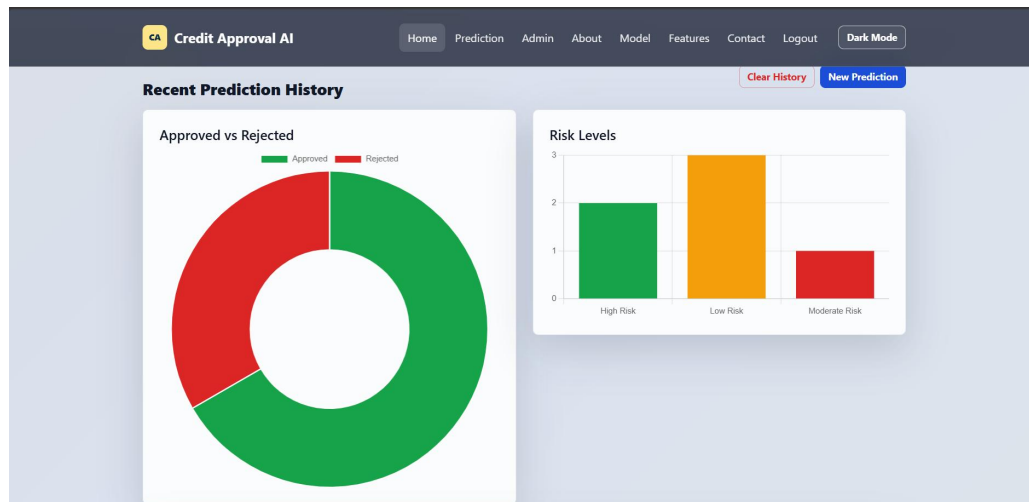


Figure 3: Confusion matrix of the selected model

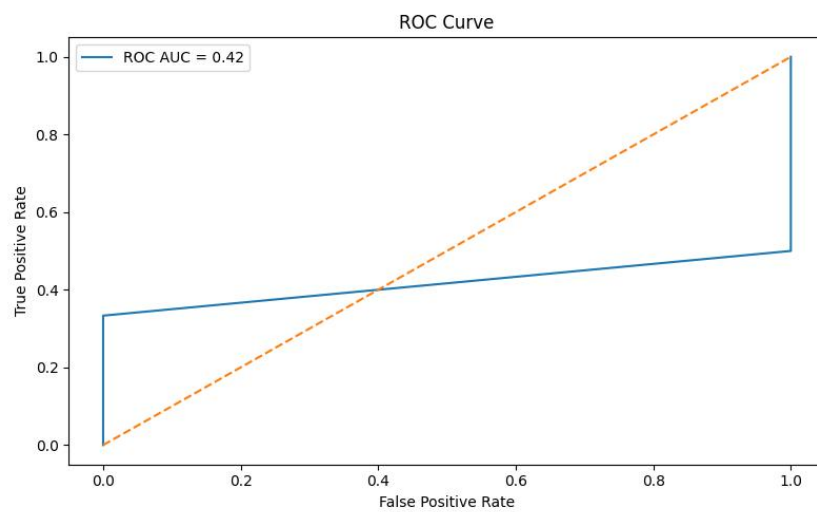
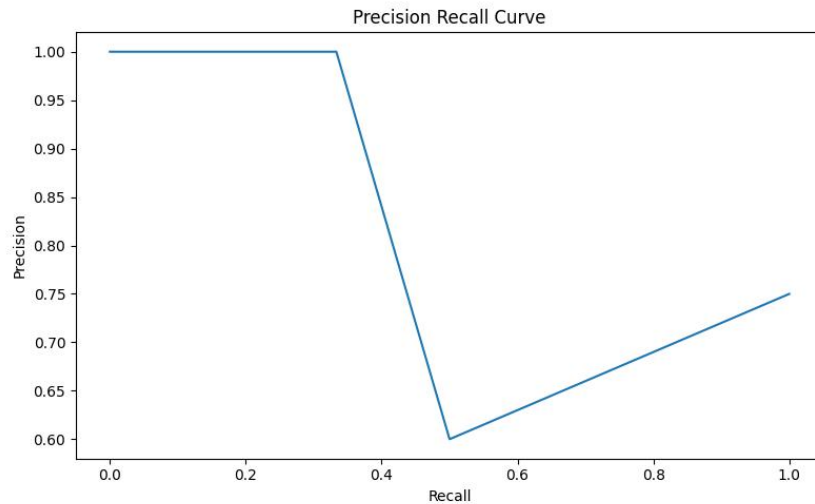


Figure 4: ROC curve of the selected model



8. Advantages & Disadvantages

Advantages

- Automates the initial credit-risk screening step, reducing manual workload.
- Compares multiple algorithms so the best-performing model is selected objectively rather than assumed.
- Supports both single-applicant and batch (CSV) predictions, covering interactive and bulk use cases.
- Provides an approval probability and risk level alongside the decision, adding transparency over a plain yes/no output.
- Includes prediction history, PDF/Excel export, and a model information page for traceability and reporting.
- Modular codebase (preprocessing.py, train.py, predict.py, app.py) makes the pipeline easy to maintain and extend.

Disadvantages

- The bundled dataset is a small sample, so reported metrics are not representative of production-scale performance and can vary between runs.
- The model has not undergone a fairness or bias audit, which is essential before any real credit decisioning use.
- Prediction history is stored in a local SQLite file rather than a production-grade database, which limits scalability.
- The current deployment is intended for demonstration; it lacks the security hardening, monitoring, and compliance controls a production system would require.

9. Conclusion

The Credit Card Approval Prediction project successfully demonstrates a complete machine learning workflow: loading and merging raw applicant and repayment data, deriving a meaningful binary target, engineering informative features, training and comparing eight classification models, and deploying the best model behind a functional Flask web application. The system supports single and batch predictions, keeps a history of past predictions, and allows results to be exported as PDF or Excel reports. Automated tests validate both the prediction logic and the surrounding application behaviour, giving confidence that the pipeline works end to end.

While the current dataset is small and the model is not yet production-ready, the project meets its stated goal: providing a working, well-documented reference implementation of the ML lifecycle that mentors, students, and analysts can inspect, run, and build upon.

10. Future Scope

- Integration with a richer, production-scale dataset to improve model reliability and generalization.
- A fairness and bias audit of the model across demographic groups before any real-world use.
- Implementation of advanced security features such as stronger authentication, data encryption, and secure transaction handling.
- Real-time inventory-style tracking is not applicable here, but real-time model monitoring and drift detection would be valuable as new data arrives.
- Migration of prediction history from SQLite to a scalable production database to support many concurrent users.
- Integration with external credit bureaus or payment gateways for richer, verified applicant data.
- Adding model explainability (the project already includes SHAP as a dependency) directly into the result page so users can see which factors drove a decision.

11. Appendix

Source Code

Key source files in the project:

- preprocessing.py – data merging, target creation, cleaning, and feature engineering.
- train.py – model training, comparison, evaluation, and artifact saving.
- predict.py – reusable prediction logic used by the web app.
- app.py – Flask application, routes, authentication, batch prediction, and history management.
- templates/ and static/ – HTML pages and styling for the web interface.
- tests/test_app.py – automated test suite.

Dataset Link

The project uses two CSV files bundled in the repository under data/: application_record.csv and credit_record.csv (based on the publicly available Credit Card Approval dataset structure).

GitHub & Project Demo Link

GitHub Repository: <https://github.com/penumakadharshini07-del/Credit-Card-Approval-Prediction>

Project Demo: <https://drive.google.com/file/d/1kIUm-oP8u1lnr80BFR3EVIItkxLrdccuK/view>